

Lab Code [10 points]
Filename: 7SegmentDisplay.sv
AndrewID: wenzhuol

```
1 `default_nettype none
2
3 // convert BCD to 7 segment format for FPGA HEX display
4 module BCDtoSevenSegment
5     (input logic [3:0] bcd,
6      output logic [6:0] segment);
7
8     always_comb begin
9         case(bcd)
10             4'd0: segment = 7'b011_1111;
11             4'd1: segment = 7'b000_0110;
12             4'd2: segment = 7'b101_1011;
13             4'd3: segment = 7'b100_1111;
14             4'd4: segment = 7'b110_0110;
15             4'd5: segment = 7'b110_1101;
16             4'd6: segment = 7'b111_1101;
17             4'd7: segment = 7'b000_0111;
18             4'd8: segment = 7'b111_1111;
19             4'd9: segment = 7'b110_1111;
20             default: segment = 7'b0;
21         endcase
22     end
23
24 endmodule: BCDtoSevenSegment
```

Lab Code [10 points]
Filename: ChipInterface.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2 // This module define the chip interface
3 // for our FPGA
4
5 module ChipInterface
6     (input  logic      CLOCK_50,
7      input  logic [3:0] KEY,
8      input  logic [17:0] SW,
9      output logic [17:0] LEDR,
10     output logic [6:0] HEX0, HEX1, HEX2, HEX3,
11     output logic [6:0] HEX4, HEX5, HEX6, HEX7,
12     output logic [7:0] VGA_R, VGA_G, VGA_B,
13     output logic      VGA_BLANK_N, VGA_CLK, VGA_SYNC_N,
14     output logic      VGA_VS, VGA_HS);
15
16     logic HS, VS, blank, reset,
17           black_screen, set_Red, set_Green, set_Blue,
18           set_Blue1, set_Blue2, set_Green1, set_Green2;
19     logic [8:0] row;
20     logic [9:0] col;
21
22     // instantiate vga module
23     vga v1(.CLOCK_50, .row, .col, .HS(VGA_HS), .VS(VGA_VS), .blank);
24
25     // Connect VGA active low signals
26     assign VGA_BLANK_N = ~blank;
27     assign VGA_SYNC_N = 1'b0;
28     assign VGA_CLK = ~CLOCK_50;
29
30     // update display every screen draw
31     logic update_display, drawn;
32     assign drawn = ((row == 479) & (col == 640));
33     smallFSM fsm (.screenDraw(drawn), .send_update(update_display),
34                  .clock(CLOCK_50), .reset);
35
36     //----- Key input section -----//
37     logic moveLeftKey, moveRightKey, serveKey, resetKey,
38           moveLeftKey_synced, moveRightKey_synced, serveKey_synced;
39
40     Synchronizer syn1 (.async(moveLeftKey), .clock(CLOCK_50),
41                      .sync(moveLeftKey_synced));
42     Synchronizer syn2 (.async(moveRightKey), .clock(CLOCK_50),
43                      .sync(moveRightKey_synced));
44     Synchronizer syn3 (.async(serveKey), .clock(CLOCK_50),
45                      .sync(serveKey_synced));
46     Synchronizer syn4 (.async(resetKey), .clock(CLOCK_50), .sync(reset));
47
48     assign moveLeftKey = ~KEY[3];
49     assign moveRightKey = ~KEY[0];
50     assign serveKey = ~KEY[1];
51     assign resetKey = ~KEY[2];
52
53     //----- HEX display section -----//
54     logic[3:0] num_lives;
55     logic[6:0] live_disp;
56     logic missed, lose_live;
57     assign missed = (ballTop > 9'd470);
58     Counter #(4) lives(.en(lose_live), .load(reset), .up(1'b1),
59                      .clock(CLOCK_50), .D(4'd3), .Q(num_lives));
60
61     BCDtoSevenSegment bss (.bcd(num_lives), .segment(live_disp));
62
63     assign HEX6 = ~live_disp;
64     // blank all other hex
65     assign HEX0 = 7'b111_1111;
66     assign HEX1 = 7'b111_1111;
67     assign HEX2 = 7'b111_1111;
68     assign HEX3 = 7'b111_1111;
69     assign HEX4 = 7'b111_1111;
```

```

70  assign HEX5 = 7'b111_1111;
71  assign HEX7 = 7'b111_1111;
72
73  //----- Brick section -----//
74  logic moving_right, moving_up;
75
76  logic atYellow, atPurple, atWall, hit;
77  logic [11:0] Q, exists;
78  logic [5:0] YArray, PArray;
79
80  yellowBricks yb (.row, .col, .reset, .clock(update_display),
81                  .atBrick(atYellow), .exists, .hit(1'b0));
82  purpleBricks pb (.row, .col, .reset, .clock(update_display),
83                  .atBrick(atPurple), .exists, .hit(1'b0));
84
85  logic [9:0] paddle_left;
86  logic atPaddle;
87  //----- FSM section -----//
88  logic lost, win, stay;
89  logic [2:0] curr_state;
90  gameFSM breakout (.hit_top, .hit_right, .serve(serveKey_synced), .lost,
91                  .win, .reset, .clock(update_display), .stay, .missed,
92                  .lose_live, .moving_right, .moving_up, .curr_state);
93  assign lost = (num_lives == 4'b0);
94  assign win = (exists == 12'd0);
95
96  assign LEDR[0] = (curr_state == 3'd0);
97  assign LEDR[1] = (curr_state == 3'd1);
98  assign LEDR[2] = (curr_state == 3'd2);
99  assign LEDR[3] = (curr_state == 3'd3);
100 assign LEDR[4] = (curr_state == 3'd4);
101 assign LEDR[5] = (curr_state == 3'd5);
102 assign LEDR[6] = (curr_state == 3'd6);
103
104 assign LEDR[10] = hit_top;
105 assign LEDR[11] = hit_right;
106
107
108 //----- Paddle section -----//
109 Paddle_counter pc (.left(moveLeftKey_synced), .right(moveRightKey_synced),
110                  .clock(CLOCK_50), .update_display, .reset, .paddle_left);
111
112 Paddle pd (.row, .col, .paddle_left, .atPaddle);
113
114 logic atBall, serve;
115 logic [8:0] ballTop;
116 logic [9:0] ballLeft;
117 Register #(1) rrr (.en(serveKey_synced), .clear(reset), .clock(CLOCK_50),
118                  .D(1'b1), .Q(serve));
119
120 //----- Ball movement section -----//
121
122 Ball_position_counter bpc (.moving_right, .moving_up, .update_display,
123                          .clock(CLOCK_50), .reset, .ballTop, .ballLeft,
124                          .serve, .paddle_left, .stay);
125 Ball ba (.row, .col, .ballTop, .ballLeft, .atBall);
126
127
128 //----- Wall section -----//
129 logic twall, lrwall;
130 Walls ws (.*);
131
132 //----- Hit detection section -----//
133
134 logic [11:0] newArray;
135 assign newArray = Q & {PArray, YArray};
136
137 always_ff begin
138   @(posedge update_display, posedge reset)
139     if (reset) exists <= 12'b1111_1111_1111;
140     else exists <= newArray;

```

```
141     end
142
143     Register #(12) r12 (.en(drawn), .clear(1'b0), .clock(CLOCK_50),
144                       .D(exists), .Q(Q));
145
146
147
148     logic hit_right, hit_top;
149     Ball_collision bcl (.clock(CLOCK_50), .ballTop, .ballLeft,
150                       .paddle_left, .exists,
151                       .hit_right, .hit_top, .PArray, .YArray);
152
153
154     //----- Color section -----//
155     assign VGA_R = atBall | atYellow | atPurple ? 8'hFF
156                   : atWall ? 8'hCC : 8'h00;
157                   // win state
158     assign VGA_G = (curr_state == 3'd6) | atBall | atPaddle | atYellow ? 8'hFF
159                   : atWall ? 8'hCC : 8'h00;
160     assign VGA_B = atBall | atPurple ? 8'hFF
161                   : atWall ? 8'hCC : 8'h00;
162
163 endmodule: ChipInterface
```

Lab Code [10 points]
Filename: ballCollision.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2 // Keep track of collision events
3 module Ball_collision
4 (
5     input logic clock,
6     input logic [11:0] exists,
7     input logic [8:0] ballTop,
8     input logic [9:0] ballLeft, paddle_left,
9     output logic hit_right, hit_left, hit_top, hit_bottom,
10    output logic [5:0] YArray, PArray
11 );
12
13 logic [8:0] ballBottom;
14 logic [9:0] ballRight;
15
16 Adder #(9) bR1(.cin(1'b0), .A(ballTop),
17               .B(9'd4), .cout(), .sum(ballBottom));
18 Adder #(10) bR2(.cin(1'b0), .A(ballLeft),
19                .B(10'd4), .cout(), .sum(ballRight));
20
21 logic atYellow, atWallTop, atPaddle, hit;
22 logic atPurple, atWallLR;
23 logic [5:0] YArray1, YArray2, PArray1, PArray2;
24 // check if the position of ball is at anywhere
25 yellowBricks yb2 (.row(ballBottom), .col(ballRight), .clock(clock),
26                  .atBrick(atYellow), .hit(1'b1), .exists, .YArray(YArray));
27 purpleBricks pb2 (.row(ballBottom), .col(ballRight), .clock(clock),
28                  .atBrick(atPurple), .hit(1'b1), .exists, .PArray(PArray));
29
30 Walls ws1 (.row(ballTop), .col(ballLeft), .atWall(),
31            .twall(atWallTop), .lrwall(atWallLR));
32 Paddle pd (.row(ballBottom), .col(ballRight),
33            .paddle_left(paddle_left), .atPaddle(atPaddle));
34
35 always_comb begin
36     // if ball at wall and ball is at top
37     hit_top = 1'b0;
38     hit_right = 1'b0;
39     if (atWallTop) begin
40         hit_top = 1'b1;
41     // if right hit the wall
42     end else if (atWallLR) begin
43         hit_right = 1'b1;
44     // if ball at brick and ball is top of brick
45     end else if (atYellow | atPurple) begin
46         if (ballTop > 9'd124 && ballTop < 9'd129
47             || ballTop > 9'd174 && ballTop < 9'd179
48             || ballTop > 9'd95 && ballTop < 9'd100
49             || ballTop > 9'd145 && ballTop < 9'd150) begin
50             hit_top = 1'b1;
51         end else begin
52             hit_right = 1'b1;
53         end
54     // if ball is at paddle and ball going down
55     end else if (atPaddle & ballTop > 9'd435 && ballTop < 9'd440) begin
56         hit_top = 1'b1;
57     end
58 end
59 endmodule: Ball_collision
```

Lab Code [10 points]
Filename: bricks.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2 // checks if the point in within the bounds of the brick
3 module Brick
4 (
5   input logic [8:0] row, brick_top, brick_height,
6   input logic [9:0] col, brick_left, brick_width,
7   input logic clock, reset, exists,
8   output logic atBrick);
9
10  logic brick_row, brick_col, exist;
11
12  offset_check #(9) of1 (.val(row), .low(brick_top), .delta(brick_height),
13    .is_between(brick_row));
14  offset_check #(10) of2 (.val(col), .low(brick_left), .delta(brick_width),
15    .is_between(brick_col));
16
17  assign atBrick = brick_col & brick_row & exists;
18
19 endmodule: Brick
20
21
22
23 // checks if the point in within the bounds of one of the brick
24 module yellowBricks
25 (input logic [11:0] exists,
26   input logic [8:0] row,
27   input logic [9:0] col,
28   input logic reset, clock, hit,
29   output logic atBrick,
30   output logic [5:0] YArray);
31
32  logic b0, b1, b2, b3, b4, b5;
33
34  // instantiate the top row
35  Brick bb0(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
36    .brick_left(10'd40), .brick_width(10'd100),
37    .atBrick(b0), .exists(exists[0]), .*);
38
39  Brick bb1(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
40    .brick_left(10'd240), .brick_width(10'd100),
41    .atBrick(b1), .exists(exists[1]), .*);
42
43  Brick bb2(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
44    .brick_left(10'd440), .brick_width(10'd100),
45    .atBrick(b2), .exists(exists[2]), .*);
46
47  Brick bb3(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
48    .brick_left(10'd90), .brick_width(10'd100),
49    .atBrick(b3), .exists(exists[3]), .*);
50
51  Brick bb4(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
52    .brick_left(10'd290), .brick_width(10'd100),
53    .atBrick(b4), .exists(exists[4]), .*);
54
55  Brick bb5(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
56    .brick_left(10'd490), .brick_width(10'd100),
57    .atBrick(b5), .exists(exists[5]), .*);
58
59  assign atBrick = (b0 | b1 | b2 | b3 | b4 | b5);
60  assign YArray = hit ? ~{b5, b4, b3, b2, b1, b0} : 6'b00_0000;
61
62 endmodule: yellowBricks
63
64 // Purple bricks module
65
66 module purpleBricks
67 (input logic [11:0] exists,
68   input logic [8:0] row,
69   input logic [9:0] col,
```

```
70 input logic reset, clock, hit,
71 output logic atBrick,
72 output logic [5:0] PArray);
73
74 logic b6, b7, b8, b9, b10, b11;
75
76 Brick bb6(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
77           .brick_left(10'd140), .brick_width(10'd100),
78           .atBrick(b6), .exists(exists[6]), .*);
79
80 Brick bb7(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
81           .brick_left(10'd340), .brick_width(10'd100),
82           .atBrick(b7), .exists(exists[7]), .*);
83
84 Brick bb8(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
85           .brick_left(10'd540), .brick_width(10'd50),
86           .atBrick(b8), .exists(exists[8]), .*);
87
88 // instantiate the bottom row
89 Brick bb9(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
90           .brick_left(10'd40), .brick_width(10'd50),
91           .atBrick(b9), .exists(exists[9]), .*);
92
93 Brick bb10(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
94            .brick_left(10'd190), .brick_width(10'd100),
95            .atBrick(b10), .exists(exists[10]), .*);
96
97 Brick bb11(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
98            .brick_left(10'd390), .brick_width(10'd100),
99            .atBrick(b11), .exists(exists[11]), .*);
100
101 assign atBrick = (b6 | b7 | b8 | b9 | b10 | b11);
102 assign PArray = hit ? ~{b11, b10, b9, b8, b7, b6} : 6'b00_0000;
103
104 endmodule: purpleBricks
```

Lab Code [10 points]
Filename: bricks_old.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2 // checks if the point is within the bounds of the brick
3 module Brick
4   (input logic [8:0] row, ballRow, brick_top, brick_height,
5    input logic [9:0] col, ballCol, brick_left, brick_width,
6    input logic clock, reset,
7    output logic atBrick);
8
9   logic brick_row, brick_col, exist;
10
11   offset_check #(9) of1 (.val(row), .low(brick_top), .delta(brick_height),
12                        .is_between(brick_row));
13   offset_check #(10) of2 (.val(col), .low(brick_left), .delta(brick_width),
14                        .is_between(brick_col));
15
16   // brick ~exist if hit
17   logic gone, colHit, rowHit, hit;
18   assign atBrick = brick_row & brick_col & exist;
19
20   offset_check #(9) of3 (.val(ballRow), .low(brick_top), .delta(brick_height),
21                        .is_between(rowHit));
22   offset_check #(10) of4 (.val(ballCol), .low(brick_left), .delta(brick_width),
23                        .is_between(colHit));
24
25   assign hit = rowHit & colHit;
26
27   CustomRegister #(1) cr (.en(reset), .clear(hit), .clock, .D(1'b1), .Q(exist));
28
29 endmodule: Brick
30
31 // Custom Register
32 // clear > enable
33 module CustomRegister
34   #(parameter WIDTH = 8)
35   (input logic en, clear, clock,
36    input logic [WIDTH-1:0] D,
37    output logic [WIDTH-1:0] Q);
38
39   always_ff @(posedge clock)
40     if (clear) Q <= 0;
41     else if (en) Q <= D;
42
43 endmodule: CustomRegister
44
45 // checks if the point is within the bounds of one of the bricks
46 module yellowBricks
47   (input logic [8:0] row, ballRow,
48    input logic [9:0] col, ballCol,
49    input logic reset, clock,
50    output logic atBrick);
51
52   logic b0, b1, b2, b3, b4, b5;
53
54   // instantiate the top row
55   Brick bb0(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
56            .brick_left(10'd40), .brick_width(10'd100), .atBrick(b0), .*);
57
58   Brick bb1(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
59            .brick_left(10'd240), .brick_width(10'd100), .atBrick(b1), .*);
60
61   Brick bb2(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
62            .brick_left(10'd440), .brick_width(10'd100), .atBrick(b2), .*);
63
64   Brick bb3(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
65            .brick_left(10'd90), .brick_width(10'd100), .atBrick(b3), .*);
66
67   Brick bb4(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
68            .brick_left(10'd290), .brick_width(10'd100), .atBrick(b4), .*);
69
```



```
70  Brick bb5(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
71            .brick_left(10'd490), .brick_width(10'd100), .atBrick(b5), .*);
72
73  assign atBrick = (b0 | b1 | b2 | b3 | b4 | b5);
74
75  endmodule: yellowBricks
76
77  module purpleBricks
78  (input logic [8:0] row, ballRow,
79   input logic [9:0] col, ballCol,
80   input logic reset, clock, hit,
81   output logic atBrick);
82
83   logic b6, b7, b8, b9, b10, b11;
84
85   Brick bb6(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
86            .brick_left(10'd140), .brick_width(10'd100), .atBrick(b6), .*);
87
88   Brick bb7(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
89            .brick_left(10'd340), .brick_width(10'd100), .atBrick(b7), .*);
90
91   Brick bb8(.row(row), .col(col), .brick_top(9'd100), .brick_height(9'd30),
92            .brick_left(10'd540), .brick_width(10'd50), .atBrick(b8), .*);
93
94   // instantiate the botton row
95   Brick bb9(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
96            .brick_left(10'd40), .brick_width(10'd50), .atBrick(b9), .*);
97
98   Brick bb10(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
99            .brick_left(10'd190), .brick_width(10'd100), .atBrick(b10), .*);
100
101   Brick bb11(.row(row), .col(col), .brick_top(9'd150), .brick_height(9'd30),
102            .brick_left(10'd390), .brick_width(10'd100), .atBrick(b11), .*);
103
104
105   assign atBrick = (b6 | b7 | b8 | b9 | b10 | b11);
106
107  endmodule: purpleBricks
```

Lab Code [10 points]
Filename: counters.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2
3 // Module to keep track of score
4 // stop when game end
5 module ScoreCount
6     (input logic [8:0] ball_top,
7      input logic load,
8      output logic [1:0] lives,
9      output logic lost);
10
11     logic die;
12     // Lose live when bottom hit
13     MagComp #(9) Comp_Die (.A(9'd475), .B(ball_top), .AeqB(die));
14
15     Counter #(2) liveCount (.en(die), .clear(1'b0),
16                             .up(1'b0), .load, .D(2'd3), .Q(lives));
17     MagComp #(2) gameLost (.A(lives), .B(2'd0), .AeqB(lost));
18
19 endmodule: ScoreCount
20
21 // two counters to keep track of ball
22 module Ball_position_counter
23     (input logic moving_right, moving_up, update_display,
24      clock, reset, serve, stay,
25      input logic [9:0] paddle_left,
26      output logic [8:0] ballTop,
27      output logic [9:0] ballLeft);
28     // flipped because row counts from the top
29     logic moving_up_N, moveBall, putBack, putBack2;
30     assign moving_up_N = ~moving_up;
31     assign moveBall = update_display & serve & ~stay;
32     assign putBack = reset | stay;
33     Counter_by #(9) top(.en(moveBall), .load(putBack), .up(moving_up_N),
34                        .by(9'd2), .clock, .D(9'd435), .Q(ballTop));
35     logic [9:0] startballLeft, paddle_center;
36     assign paddle_center = paddle_left + 10'd32;
37
38     // stick with paddle if the game has not started
39
40     assign ballLeft = (serve & ~stay) ? startballLeft : paddle_center;
41     assign putBack2 = (~serve | stay);
42     // keep loading current paddle position until game start
43     Counter #(10) left(.en(moveBall), .load(putBack2), .up(moving_right),
44                       .clock, .D(paddle_center), .Q(startballLeft));
45
46 endmodule: Ball_position_counter
47
48
49 // counter to keep track of paddle position
50 module Paddle_counter
51     (input logic left, right, clock,
52      update_display, reset,
53      output logic [9:0] paddle_left);
54
55     logic move, valid_input, can_move, can_move_left, can_move_right;
56
57     assign valid_input = left ^ right; // only if one of the button is pressed
58     assign can_move = (left & can_move_left) | (right & can_move_right);
59     assign move = update_display & can_move & valid_input;
60
61     MagComp #(10) mc1 (.A(paddle_left), .B(10'd42), .AgtB(can_move_left));
62     MagComp #(10) mc2 (.A(paddle_left), .B(10'd523), .AltB(can_move_right));
63
64     Counter_by #(10) pd(.en(move), .load(reset), .up(right), .clear(1'b0),
65                        .by(10'd5), .clock, .D(10'd287), .Q(paddle_left));
66
67 endmodule: Paddle_counter
68
69 // counter that can count by arbitrary amount
```

```
70 module Counter_by
71     #(parameter WIDTH = 8)
72     (input logic en, clear, load, up, clock,
73      input logic [WIDTH-1:0] by,
74      input logic [WIDTH-1:0] D,
75      output logic [WIDTH-1:0] Q);
76
77     always_ff @(posedge clock)
78         if (clear) Q <= 0;
79         else if (load) Q <= D;
80         else if (en) begin
81             if (up) Q <= Q + by;
82             else Q <= Q - by;
83         end
84
85 endmodule: Counter_by
```

Lab Code [10 points]
Filename: game_state.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2 // make sure screen update is only asserted for 1 clock cycle
3 module smallFSM
4   (input logic screenDraw, clock, reset,
5     output logic send_update);
6
7   enum logic {NOT_UPDATE, UPDATE} state, nextState;
8
9   // next state logic
10  always_comb begin
11    nextState = NOT_UPDATE;
12    case (state)
13      NOT_UPDATE:
14        nextState = (screenDraw) ? UPDATE : NOT_UPDATE;
15      UPDATE:
16        nextState = NOT_UPDATE;
17    endcase
18  end
19
20  // output logic
21  always_comb begin
22    unique case (state)
23      NOT_UPDATE: send_update = 0;
24      UPDATE: send_update = 1;
25    endcase
26  end
27
28  always_ff @(posedge clock)
29    if (reset)
30      state <= NOT_UPDATE;
31    else
32      state <= nextState;
33
34 endmodule: smallFSM
35
36 // main game control FSM
37 module gameFSM
38   (input logic hit_top, hit_right, serve, lost,
39     win, reset, clock, missed,
40     output logic stay, moving_right, moving_up, lose_live,
41     output logic [2:0] curr_state);
42
43   enum logic [2:0] {WAIT, MOVINGTOPLEFT,
44                     MOVINGTOPRIGHT, MOVINGBOTTOMLEFT,
45                     MOVINGBOTTOMRIGHT, WIN, LOST} state, nextState;
46
47   // Next state logic here
48   always_comb begin
49     nextState = WAIT; // default state
50     case (state)
51       // Initial state wait for game to begin
52       WAIT:
53         nextState = (serve) ? MOVINGTOPRIGHT : WAIT;
54       MOVINGTOPLEFT:
55         if (hit_right) begin
56           nextState = MOVINGTOPRIGHT;
57         end else if (hit_top) begin
58           nextState = MOVINGBOTTOMLEFT;
59         end else begin
60           nextState = (reset) ? WAIT : (lost)
61             ? LOST : (win) ? WIN : MOVINGTOPLEFT;
62         end
63       MOVINGTOPRIGHT:
64         if (hit_right) begin
65           nextState = MOVINGTOPLEFT;
66         end else if (hit_top) begin
67           nextState = MOVINGBOTTOMRIGHT;
68         end else begin
69           nextState = (reset) ? WAIT : (lost)
```

```

70         ? LOST : (win) ? WIN : MOVINGTOPRIGHT;
71     end
72     MOVINGBOTTOMLEFT:
73         if (missed) nextState = WAIT;
74         else if (hit_right) begin
75             nextState = MOVINGBOTTOMRIGHT;
76         end else if (hit_top) begin
77             nextState = MOVINGTOPLEFT;
78         end else begin
79             nextState = (reset) ? WAIT : (lost)
80             ? LOST : (win) ? WIN : MOVINGBOTTOMLEFT;
81         end
82     MOVINGBOTTOMRIGHT:
83         if (missed) nextState = WAIT;
84         else if (hit_right) begin
85             nextState = MOVINGBOTTOMLEFT;
86         end else if (hit_top) begin
87             nextState = MOVINGTOPRIGHT;
88         end else begin
89             nextState = (reset) ? WAIT : (lost)
90             ? LOST : (win) ? WIN : MOVINGBOTTOMRIGHT;
91         end
92     LOST:
93         nextState = (reset) ? WAIT : LOST;
94     WIN:
95         nextState = (reset) ? WAIT : WIN;
96     endcase
97 end
98
99
100 // Output logic here
101 always_comb begin
102     unique case (state)
103         WAIT: begin
104             stay = 1'b1;
105             moving_right = 1'b1;
106             moving_up = 1'b1;
107             curr_state = 3'd0;
108             lose_live = 1'b0;
109         end
110         MOVINGTOPRIGHT: begin
111             moving_right = 1'b1;
112             moving_up = 1'b1;
113             stay = 1'b0;
114             curr_state = 3'd1;
115             lose_live = 1'b0;
116         end
117         MOVINGTOPLEFT: begin
118             moving_right = 1'b0;
119             moving_up = 1'b1;
120             stay = 1'b0;
121             curr_state = 3'd2;
122             lose_live = 1'b0;
123         end
124         MOVINGBOTTOMLEFT: begin
125             moving_right = 1'b0;
126             moving_up = 1'b0;
127             stay = 1'b0;
128             curr_state = 3'd3;
129             lose_live = (missed) ? 1'b1 : 1'b0;
130         end
131         MOVINGBOTTOMRIGHT: begin
132             moving_right = 1'b1;
133             moving_up = 1'b0;
134             stay = 1'b0;
135             curr_state = 3'd4;
136             lose_live = (missed) ? 1'b1 : 1'b0;
137         end
138
139         LOST: begin
140             stay = 1'b1;

```

```
141         moving_right = 1'b1;
142         moving_up = 1'b1;
143         curr_state = 3'd5;
144         lose_live = 1'b0;
145     end
146
147     WIN: begin
148         stay = 1'b1;
149         moving_right = 1'b1;
150         moving_up = 1'b1;
151         curr_state = 3'd6;
152         lose_live = 1'b0;
153     end
154 endcase
155 end
156
157
158 // Synchronous state update described here as an always_ff block.
159 // In essence, these flip flops that will hold the state
160 // synchronous reset
161
162 always_ff @(posedge clock, posedge reset)
163     if (reset)
164         state <= WAIT;
165     else
166         state <= nextState;
167
168 endmodule: gameFSM
```

Lab Code [10 points]
Filename: lab04.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2
3 module vga_test();
4
5     logic CLOCK_50, reset, HS, VS, blank;
6     logic [10:0] num_1600_periods;
7     logic [8:0] row;
8     logic [9:0] col;
9
10    initial begin
11        CLOCK_50 = 0;
12        forever #1 CLOCK_50 = ~CLOCK_50;
13    end
14
15    vga DUT(.*);
16
17    initial begin
18        reset = 1;
19        #1 reset <= 0;
20        #4000000;
21
22        #1 $finish;
23    end
24
25 endmodule: vga_test
```

Lab Code [10 points]
Filename: library.sv
AndrewID: wenzhuol

```
1 `default_nettype none
2 // A library of basic logical components
3 /*
4  * Multiple testbenches and initial blocks exist.
5  * use vlogan -sverilog -nc library.sv library_tests.sv
6  *     vcs -sverilog -nc MODULE_NAME
7  *     ./simv
8  */
9
10 //-----HW6 Updated-----//
11
12 // Assert lt, eq, or gt for two inputs
13 // Parameterized width
14 module MagComp
15     #(parameter WIDTH = 8)
16     (input logic [WIDTH-1:0] A, B,
17      output logic AltB, AeqB, AgtB);
18
19     assign AltB = (A < B);
20     assign AeqB = (A == B);
21     assign AgtB = (A > B);
22
23 endmodule: MagComp
24
25 // Full Adder with carry-out
26 // Parameterized width
27 module Adder
28     #(parameter WIDTH = 8)
29     (input logic      cin,
30      input logic [WIDTH-1:0] A, B,
31      output logic      cout,
32      output logic [WIDTH-1:0] sum);
33
34     assign {cout, sum} = A + B + cin;
35
36 endmodule: Adder
37
38 // Parameterized width multiplexer
39 module Multiplexer
40     #(parameter WIDTH = 8)
41     (input logic [WIDTH-1:0] I,
42      input logic [$clog2(WIDTH)-1:0] S, // Ceiling of log2 of width
43      output logic Y);
44
45     assign Y = I[S];
46
47 endmodule: Multiplexer
48
49 // Choose 1 from 2 multi-bit inputs
50 module Mux2to1
51     #(parameter WIDTH = 8)
52     (input logic [WIDTH-1:0] I0, I1,
53      input logic S,
54      output logic [WIDTH-1:0] Y);
55
56     assign Y = (S) ? I1 : I0;
57
58 endmodule: Mux2to1
59
60 // Decode an n-bit input to one hot encoding output
61 module Decoder
62     #(parameter WIDTH = 8)
63     (input logic [$clog2(WIDTH)-1:0] I,
64      input logic en,
65      output logic [WIDTH-1:0] D);
66
67     always_comb begin
68         D = 'd0; // All bits set to zero
69         if (en) D[I] = 1'b1;
```



```
70     else D[I] = 1'b0;
71   end
72
73 endmodule: Decoder
74
75 // DFlipFlop with async active-low preset and reset
76 module DFlipFlop
77   (input  logic D,
78    input  logic preset_L, reset_L, clock,
79    output logic Q);
80
81   always_ff @(posedge clock, negedge preset_L, negedge reset_L)
82     if (~preset_L)
83       Q <= 1'b1;
84     else if (~reset_L)
85       Q <= 1'b0;
86     else
87       Q <= D;
88
89 endmodule : DFlipFlop
90
91 // Parameterized width Register
92 // enable > clear
93 module Register
94   #(parameter WIDTH = 8)
95   (input logic en, clear, clock,
96    input logic [WIDTH-1:0] D,
97    output logic [WIDTH-1:0] Q);
98
99   always_ff @(posedge clock)
100     if (en) Q <= D;
101     else if (clear) Q <= 0;
102
103 endmodule: Register
104
105 // Counter with two direction of counting
106 // clear > load > enable > counting
107 module Counter
108   #(parameter WIDTH = 8)
109   (input logic en, clear, load, up, clock,
110    input logic [WIDTH-1:0] D,
111    output logic [WIDTH-1:0] Q);
112
113   always_ff @(posedge clock)
114     if (clear) Q <= 0;
115     else if (load) Q <= D;
116     else if (en) begin
117       if (up) Q <= Q + 1;
118       else Q <= Q - 1;
119     end
120
121 endmodule: Counter
122
123 // 2-FF synchronizer
124 module Synchronizer
125   (input  logic async, clock,
126    output logic sync);
127
128   logic metastable;
129
130   DFlipFlop one(.D(async),
131                .Q(metastable),
132                .clock,
133                .preset_L(1'b1),
134                .reset_L(1'b1)
135                );
136
137   DFlipFlop two(.D(metastable),
138                .Q(sync),
139                .clock,
140                .preset_L(1'b1),
```

```

141         .reset_L(1'b1)
142     );
143
144 endmodule : Synchronizer
145
146 // Serial-in-parallel-out Shift Register
147 // left to indicate direction of shift
148 module ShiftRegister_SIPO
149     #(parameter WIDTH = 8)
150     (input logic serial, en, left, clock,
151      output logic [WIDTH-1:0] Q);
152
153     logic [WIDTH-1:0] Q_shifted;
154
155     always_comb begin
156         if (left)
157             Q_shifted = {Q[WIDTH-2:0], serial};
158         else
159             Q_shifted = {serial, Q[WIDTH-1:1]};
160     end
161
162     always_ff @(posedge clock)
163         if (en) Q <= Q_shifted;
164         else Q <= Q;
165
166 endmodule: ShiftRegister_SIPO
167
168 // Parallel-in-parallel-out Shift Register
169 // left to indicate direction of shift
170 module ShiftRegister_PIPO
171     #(parameter WIDTH = 8)
172     (input logic en, left, load, clock,
173      input logic [WIDTH-1:0] D,
174      output logic [WIDTH-1:0] Q);
175
176     logic [WIDTH-1:0] Q_shifted;
177     assign Q_shifted = (left) ? Q << 1 : Q >> 1;
178
179     always_ff @(posedge clock)
180         if (load) Q <= D;
181         else if (en) Q <= Q_shifted;
182
183 endmodule: ShiftRegister_PIPO
184
185 // A PIPO register that shift left
186 // by N positions indicated by a 2-bit input
187 module BarrelShiftRegister
188     #(parameter WIDTH = 8)
189     (input logic en, load, clock,
190      input logic [1:0] by,
191      input logic [WIDTH-1:0] D,
192      output logic [WIDTH-1:0] Q);
193
194     always_ff @(posedge clock)
195         if (load) Q <= D;
196         else if (en) Q <= Q << by;
197
198 endmodule: BarrelShiftRegister
199
200 // BusDriver control access to a bus,
201 // drive data when enabled, otherwise high-impedance
202 module BusDriver
203     #(parameter WIDTH = 8)
204     (input logic en,
205      input logic [WIDTH-1:0] data,
206      output logic [WIDTH-1:0] buff,
207      inout tri [WIDTH-1:0] bus);
208
209     assign buff = bus;
210     assign bus = (en) ? data : 'bz; // High-impedance if not enabled
211

```

```
212 endmodule: BusDriver
213
214 // Memory module with a shared read/write bus
215 module Memory
216     #(parameter DW = 16, // Word size
217         W = 256, // Num words
218         AW = $clog2(W) // Address Width
219     (input logic re, we, clock,
220     input logic [AW-1:0] addr,
221     inout tri [DW-1:0] data);
222
223     logic [DW-1:0] M[W];
224     logic [DW-1:0] rData;
225
226     assign rData = M[addr];
227     // drive when read-enabled
228     assign data = (re) ? rData : 'bz;
229
230     always_ff @(posedge clock)
231         if (we)
232             M[addr] <= data;
233
234 endmodule: Memory
235
236 // check if low <= val <= high
237 module range_check
238     #(parameter WIDTH = 8)
239     (input logic [WIDTH-1:0] val, low, high,
240     output logic is_between);
241
242     assign is_between = (val <= high) & (val >= low);
243
244 endmodule: range_check
245
246 // check if low <= val <= low + delta
247 module offset_check
248     #(parameter WIDTH = 8)
249     (input logic [WIDTH-1:0] val, low, delta,
250     output logic is_between);
251
252     assign is_between = (val >= low) & (val <= low + delta);
253
254 endmodule: offset_check
```

Lab Code [10 points]

Filename: library_tests.sv

AndrewID: wenzhuo1

```
1 `default_nettype none
2 // Testbench for all modules in library.sv
3 /*
4  * Multiple testbenches and initial blocks exist.
5  * use vlogan -sverilog -nc library.sv library_tests.sv
6  *     vcs -sverilog -nc MODULE_NAME
7  *     ./simv
8  */
9
10 // 30-bit test for MagComp
11 module MagComp_test;
12     logic [29:0] A, B;
13     logic AltB, AeqB, AgtB;
14
15     MagComp #(30) d(.);
16
17     initial begin
18         $monitor($time,,
19             "A(%d), (%d), lt/eq/gt (%b%b%b)",
20             A, B, AltB, AeqB, AgtB);
21         A = 30'd5; B = 30'd8; // lt
22         #1 A = 30'd10; B = 30'd5; // gt
23         #1 A = 30'd29; B = 30'd29; // eq
24
25         #1 $finish;
26     end
27
28 endmodule: MagComp_test
29
30 // Test 20-bit Full Adder with carry-in's
31 module Adder_test;
32     logic cin;
33     logic [19:0] A, B;
34     logic cout;
35     logic [19:0] sum;
36
37     Adder #(20) add1(.);
38     initial begin
39         $monitor($time,,
40             "A = %d, B = %d, cin = %b, cout = %b, sum = %d",
41             A, B, cin, cout, sum);
42
43         A = 20'd5;
44         B = 20'd8;
45         cin = 1'b0;
46         // ans = 13
47         #1 A = 20'd1;
48         B = 20'd1;
49         // ans = 2
50         #1 A = 20'd20;
51         B = 20'd20;
52         cin = 1'b1;
53         // ans = 41
54         #1 A = 20'd5;
55         B = 20'd0;
56         cin = 1'b0;
57         // ans = 5
58         #1 A = 20'd0;
59         B = 20'd0;
60         cin = 1'b1;
61         // ans = 1
62         #1 A = 20'd1;
63         B = 20'd2;
64         cin = 1'b0;
65         // ans = 3
66         #1 $finish;
67     end
68
69 endmodule: Adder_test
```

```

70
71
72
73 // Test all 6 select inputs for 6-bit multiplexer
74 module Multiplexer_test;
75     logic [5:0] I;
76     logic [2:0] S;
77     logic Y;
78
79     Multiplexer #(6) m(.);
80     initial begin
81         $monitor($time,,
82             "I = %b, S = %b, Y = %b",
83             I, S, Y);
84
85         I = 6'b01_0010;
86         S = 3'b000; // 000 to 101
87         #1 S = 3'b001;
88         #1 S = 3'b010;
89         #1 S = 3'b011;
90         #1 S = 3'b100;
91         #1 S = 3'b101;
92
93         #1 $finish;
94     end
95
96 endmodule: Multiplexer_test
97
98 // Test select 0 and select 1
99 // for a multi-bit two-to-one multiplexer
100 module Mux2to1_test;
101     logic [15:0] I0, I1;
102     logic S;
103     logic [15:0] Y;
104
105     Mux2to1 #(16) m2to1(.);
106     initial begin
107         $monitor($time,,
108             "I0 = %d, I1 = %d, S = %b, Y = %d",
109             I0, I1, S, Y);
110         I0 = 16'd23;
111         I1 = 16'd47;
112         S = 0;
113         #1 S = 1;
114         #1 $finish;
115     end
116
117 endmodule: Mux2to1_test
118
119 // Test 5-bit input decoder
120 module Decoder_test;
121     logic [4:0] I;
122     logic en;
123     logic [31:0] D;
124
125     Decoder #(32) d(.);
126
127     initial begin
128         $monitor($time,,
129             "I = %b, en = %b, D = %b",
130             I, en, D);
131         en = 1; // Enabled
132         for (I = 5'b0; I < 5'b1_1111; I++)
133             #1;
134         #1;
135         en = 0; // Not enabled
136         for (I = 5'b0; I < 5'b1_1111; I++)
137             #1;
138         #1 $finish;
139     end
140

```

```
141 endmodule: Decoder_test
142
143 // Test functions of a DFlipFlop
144 module DFlipFlop_test;
145     logic D, Q, clock, preset_L, reset_L;
146
147     DFlipFlop dff (.*);
148
149     initial begin
150         clock = 0;
151         reset_L = 0;
152         preset_L <= 1;
153         reset_L <= 1;
154         forever #5 clock = ~clock;
155     end
156     initial begin
157         $monitor($time,,
158             "D(%b), Q(%b), clock %b, preset_L(%b), reset_L(%b)",
159             D, Q, clock, preset_L, reset_L);
160         D <= 1'b1;
161         @(posedge clock);
162         @(posedge clock);
163         @(posedge clock);
164         @(posedge clock);
165         reset_L <= 0;        // test reset_L
166         #1 reset_L <= 1;
167         @(posedge clock);
168         @(posedge clock);
169         D <= 1'b0;
170         @(posedge clock);
171         #1 preset_L <= 0;    // test preset_L
172         #1 preset_L <= 1;
173         @(posedge clock);
174         @(posedge clock);
175         #1 $finish;
176     end
177 endmodule: DFlipFlop_test
178
179 // Test en and clear of a 24-bit Register
180 module Register_test;
181     logic en, clear, clock;
182     logic [23:0] D, Q;
183
184     Register #(24) r1 (.*);
185
186     initial begin
187         clock = 0;
188         clear = 1;
189         clear <= 0;
190         en = 1;
191         forever #5 clock = ~clock;
192     end
193     initial begin
194         $monitor($time,,
195             "D(%d), Q(%d), clock %b, clear(%b), enable(%b)",
196             D, Q, clock, clear, en);
197         D <= 24'd22;
198         @(posedge clock);
199         @(posedge clock);
200         @(posedge clock);
201         en <= 0;        // test ~enable
202         #1 clear <= 1;    // test clear
203         @(posedge clock);
204         @(posedge clock);
205         clear <= 0;
206         #1 $finish;
207     end
208 endmodule: Register_test
```

```
212
213 // Test counting up and down a 9-bit Counter
214 module Counter_test;
215     logic en, clear, load, up, clock;
216     logic [8:0] D, Q;
217
218     Counter #(9) c1 (.*);
219
220     initial begin
221         clock = 0;
222         clear = 1;
223         clear <= 0;
224         load = 0;
225         up <= 1;
226         en = 1;
227         forever #5 clock = ~clock;
228     end
229
230     initial begin
231         $monitor($time,,
232             "D(%d) Q(%d) clock %b load(%b) clear(%b) up(%b) enable(%b)",
233             D, Q, clock, load, clear, up, en);
234         D <= 9'd7;
235         @(posedge clock);
236         load <= 1;
237         @(posedge clock);
238         load <= 0;
239         @(posedge clock);
240         @(posedge clock);
241         @(posedge clock);
242         @(posedge clock);
243         en <= 0;          // test ~enable
244         @(posedge clock);
245         #1 en <= 1;
246         #1 clear <= 1;    // test enable clear
247         @(posedge clock);
248         clear <= 0;
249         @(posedge clock);
250         @(posedge clock);
251         @(posedge clock);
252         @(posedge clock);
253         @(posedge clock);
254         up <= 0;
255         @(posedge clock);
256         @(posedge clock);
257         @(posedge clock);
258         #1 $finish;
259     end
260
261 endmodule: Counter_test
262
263
264 // Test ShiftRegister_SIPO of 16-bit wide
265 module ShiftRegister_SIPO_test;
266     logic serial, en, left, clock;
267     logic [15:0] Q;
268
269     ShiftRegister_SIPO #(16) sipo1 (.*);
270
271     initial begin
272         clock = 0;
273         left = 1;
274         en = 1;
275         serial = 1;
276         forever #5 clock = ~clock;
277     end
278
279     initial begin
280         $monitor($time,,
281             "serial %b en %b left %b clock %b Q %b",
282             serial, en, left, clock, Q);
```

```

283     @(posedge clock);
284     @(posedge clock);
285     @(posedge clock);
286     @(posedge clock);
287     @(posedge clock);
288     @(posedge clock);
289     serial <= 0;
290     @(posedge clock);
291     @(posedge clock);
292     @(posedge clock);
293     @(posedge clock);
294     left <= 0; // Change direction
295     @(posedge clock);
296     @(posedge clock);
297     @(posedge clock);
298     serial <= 1;
299     @(posedge clock);
300     @(posedge clock);
301     @(posedge clock);
302     @(posedge clock);
303     serial <= 0;
304     @(posedge clock);
305     @(posedge clock);
306     @(posedge clock);
307     @(posedge clock);
308     @(posedge clock);
309     serial <= 1;
310     @(posedge clock);
311     @(posedge clock);
312     @(posedge clock);
313     @(posedge clock);
314
315     #1 $finish;
316 end
317
318 endmodule: ShiftRegister_SIPO_test
319
320 // Test ShiftRegister_PIPO of 12-bit wide
321 module ShiftRegister_PIPO_test;
322     logic en, left, load, clock;
323     logic [11:0] D, Q;
324
325     ShiftRegister_PIPO #(12) pipo1 (.*);
326
327     initial begin
328         clock = 0;
329         left = 1;
330         en = 1;
331         load = 0;
332         D = 12'b1000_0110_1001;
333         forever #5 clock = ~clock;
334     end
335
336     initial begin
337         $monitor($time,,
338             "en %b left %b load %b clock %b D %b Q %b",
339             en, left, load, clock, D, Q);
340         @(posedge clock);
341         load <= 1;
342         @(posedge clock);
343         load <= 0;
344         @(posedge clock);
345         @(posedge clock);
346         @(posedge clock);
347         @(posedge clock);
348
349         @(posedge clock);
350         @(posedge clock);
351         @(posedge clock);
352         @(posedge clock);
353         left <= 0; // Change direction

```



```
354     @(posedge clock);
355     @(posedge clock);
356     @(posedge clock);
357     D <= 12'b1111_0000_1111;
358     load <= 1;          // Load new value
359     @(posedge clock);
360     load <= 0;
361     @(posedge clock);
362     @(posedge clock);
363     @(posedge clock);
364     @(posedge clock);
365     @(posedge clock);
366     @(posedge clock);
367     @(posedge clock);
368     @(posedge clock);
369     @(posedge clock);
370     @(posedge clock);
371     @(posedge clock);
372     @(posedge clock);
373
374     #1 $finish;
375 end
376
377 endmodule: ShiftRegister_PIP0_test
378
379 // Test BarrelShiftRegister of 10-bit wide
380 module BarrelShiftRegister_test;
381     logic en, load, clock;
382     logic [1:0] by;
383     logic [9:0] D, Q;
384
385     BarrelShiftRegister #(10) bsr1 (.*);
386
387     initial begin
388         clock = 0;
389         en = 1;
390         load = 0;
391         by = 2'd2;
392         D = 10'b11_0110_1111;
393         forever #5 clock = ~clock;
394     end
395
396     initial begin
397         $monitor($time,,
398             "en %b load %b clock %b by %d D %b Q %b",
399             en, load, clock, by, D, Q);
400         @(posedge clock);
401         load <= 1;
402         @(posedge clock);
403         load <= 0;
404         @(posedge clock);
405         @(posedge clock);
406         @(posedge clock);
407         @(posedge clock);
408
409         @(posedge clock);
410         @(posedge clock);
411         @(posedge clock);
412         @(posedge clock);
413         load <= 1;
414         by <= 1 ;
415         @(posedge clock);
416         load <= 0;
417         @(posedge clock);
418         @(posedge clock);
419         D <= 12'b1111_0000_1111;
420         load <= 1;          // Load new value
421         @(posedge clock);
422         load <= 0;
423         @(posedge clock);
424         @(posedge clock);
```

```
425         @(posedge clock);
426         @(posedge clock);
427         @(posedge clock);
428         @(posedge clock);
429         @(posedge clock);
430         @(posedge clock);
431         @(posedge clock);
432         @(posedge clock);
433         @(posedge clock);
434         @(posedge clock);
435
436         #1 $finish;
437     end
438
439 endmodule: BarrelShiftRegister_test
440
441 // Test memory module
442
443
444 module Memory_test;
445     logic re, we, clock;
446     logic [3:0] addr;
447     tri [7:0] data;
448     logic [7:0] data_write;
449
450     assign data = (re) ? 'bz : data_write;
451
452     Memory #(8, 16, 4) ram1(.*);
453
454     initial begin
455         clock = 0;
456         re = 0; we = 0;
457         forever #5 clock = ~clock;
458     end
459
460     initial begin
461         $monitor($time,,
462             "re %b we %b clock %b addr %d data %b",
463             re, we, clock, addr, data);
464         @(posedge clock); // Write to different addresses
465         we <= 1;
466         data_write <= 8'b1001_1010;
467         addr <= 4'd0;
468         @(posedge clock);
469         we <= 1;
470         data_write <= 8'b1111_1010;
471         addr <= 4'd3;
472         @(posedge clock);
473         @(posedge clock);
474         we <= 1;
475         data_write <= 8'b1000_0000;
476         addr <= 4'd10;
477         @(posedge clock); // Read from different addresses
478         we <= 0;
479         re <= 1;
480         addr <= 4'd3;
481         @(posedge clock);
482         @(posedge clock);
483         addr <= 4'd0;
484         @(posedge clock);
485         @(posedge clock);
486         addr <= 4'd10;
487         @(posedge clock);
488         @(posedge clock);
489         addr <= 4'd11; // Read from empty address
490         @(posedge clock);
491         @(posedge clock);
492         #1 $finish;
493     end
494 endmodule: Memory_test
495
```

```
496 // Test range_check for 16-bit inputs
497 module range_check_test;
498     logic [15:0] low, high, val;
499     logic is_between;
500
501     range_check #(16) rc(.);
502     initial begin
503         $monitor("low %d, high %d, val %d, is_between %b",
504             low, high, val, is_between);
505
506         low = 16'd16;
507         high = 16'd38;
508         val = 16'd31; // ans = 1
509         #1 val = 16'd38; // ans = 1
510         #1 val = 16'd16; // ans = 1
511         #1 low = 16'd17; // ans = 0
512         #1 val = 16'd42; // ans = 0
513
514         #1 $finish;
515     end
516 endmodule: range_check_test
517
518 // Test offset_check for 16-bit inputs
519 module offset_check_test;
520     logic [15:0] low, delta, val;
521     logic is_between;
522
523     offset_check #(16) osc(.);
524     initial begin
525         $monitor("low %d, delta %d, val %d, is_between %b",
526             low, delta, val, is_between);
527
528         low = 16'd16;
529         delta = 16'd9; // range = [16, 25]
530         val = 16'd31; // ans = 0
531         #1 val = 16'd38; // ans = 0
532         #1 val = 16'd16; // ans = 1
533         #1 low = 16'd17; // range = [17, 26] ans = 0
534         #1 val = 16'd24; // ans = 1
535         #1 val = 16'd26; // ans = 1
536
537         #1 $finish;
538     end
539 endmodule: offset_check_test
```

Lab Code [10 points]
Filename: objects.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2 // checks if a point is within the bounds of a wall
3 module Walls
4   (input logic [8:0] row,
5    input logic [9:0] col,
6    output logic atWall, twall, lrwall);
7
8   logic top_wall_row, top_wall_col;
9   logic left_right_wall_row, left_wall_col, right_wall_col;
10
11  // check if rrange of the top wall
12  range_check #(9) rc1(.val(row), .low(9'd10),
13                       .high(9'd29), .is_between(top_wall_row));
14  range_check #(10) rc2(.val(col), .low(10'd20),
15                       .high(10'd609), .is_between(top_wall_col));
16
17  // check if in the range of the left or right wall
18  range_check #(9) rc3(.val(row), .low(9'd10),
19                       .high(9'd469), .is_between(left_right_wall_row));
20  range_check #(10) rc4(.val(col), .low(10'd20),
21                       .high(10'd39), .is_between(left_wall_col));
22  range_check #(10) rc5(.val(col), .low(10'd590),
23                       .high(10'd609), .is_between(right_wall_col));
24
25  assign atWall = ((top_wall_row & top_wall_col) |
26                  (left_wall_col | right_wall_col) & left_right_wall_row);
27  assign twall = (top_wall_row & top_wall_col);
28  assign lrwall = (left_wall_col | right_wall_col) & left_right_wall_row;
29
30 endmodule: Walls
31
32 // checks if the point in within the bounds of the paddple
33 // paddle_left is the leftmost point of the paddple
34 module Paddle
35   (input logic [8:0] row,
36    input logic [9:0] col, paddle_left,
37    output logic atPaddle);
38
39   logic paddle_row, paddle_col;
40
41   range_check #(9) rc6(.val(row), .low(9'd440),
42                       .high(9'd459), .is_between(paddle_row));
43   offset_check #(10) of1(.val(col), .low(paddle_left),
44                       .delta(10'd64), .is_between(paddle_col));
45
46   assign atPaddle = paddle_row & paddle_col;
47
48 endmodule:Paddle
49
50 // checks if the point in within the bounds of the ball
51 module Ball
52   (input logic [8:0] row, ballTop,
53    input logic [9:0] col, ballLeft,
54    output logic atBall);
55
56   logic ball_row, ball_col;
57
58   offset_check #(9) of2(.val(row), .low(ballTop),
59                       .delta(9'd4), .is_between(ball_row));
60   offset_check #(10) of3(.val(col), .low(ballLeft),
61                       .delta(10'd4), .is_between(ball_col));
62
63   assign atBall = ball_row & ball_col;
64
65 endmodule:Ball
66
67 // if the place in the w drawing, w is 1
68 module winState (
69   input logic [8:0] row,
```

```
70  input logic [9:0] col,
71  output logic w
72 );
73
74 always_comb begin
75  if ((row > 9'd160 && row < 9'd288 && col > 10'd256 && col < 10'd288) ||
76      (row > 9'd160 && row < 9'd288 && col > 10'd320 && col < 10'd352) ||
77      (row > 9'd160 && row < 9'd288 && col > 10'd384 && col < 10'd416) ||
78      (row > 9'd256 && row < 9'd288 && col > 10'd256 && col < 10'd416)) begin
79      w = 1'b1;
80  end else begin
81      w = 1'b0;
82  end
83
84 end
85
86 endmodule: winState
87
88 // if the place in the l drawing, l is 1
89 module loseState (
90  input logic [8:0] row,
91  input logic [9:0] col,
92  output logic l
93 );
94
95 always_comb begin
96  if ((row > 9'd160 && row < 9'd288 && col > 10'd256 && col < 10'd288) ||
97      (row > 9'd256 && row < 9'd288 && col > 10'd256 && col < 10'd532)) begin
98      l = 1'b1;
99  end else begin
100     l = 1'b0;
101  end
102
103 end
104
105 endmodule: loseState
```

Lab Code [10 points]

Filename: test_benches.sv

AndrewID: wenzhuo1

```
1 `default_nettype none
2 // Test benches below
3 // Test ball functions
4 module ball_test ();
5     logic moving_right, moving_up, update_display, clock, reset;
6     logic [8:0] ballTop;
7     logic [9:0] ballLef;
8     logic [8:0] ballTopNew;
9     logic [9:0] ballLeftNew;
10
11     Ball_position_counter bpc (.*);
12
13     initial begin
14         $monitor("lost(%d), lives(%d)", ballLef, ballTop);
15         ball_top = 9'd150;
16         load = 1'b1
17         @(posedge clock);
18         @(posedge clock);
19         ball_top = 9'd450;
20         @(posedge clock);
21         @(posedge clock);
22         @(posedge clock);
23     end
24
25 endmodule: ball_test
26
27 // Test score count functions
28 module ScoreCount_test ();
29     logic [8:0] ball_top;
30     logic load;
31     logic [1:0] lives;
32     logic lost;
33
34     initial begin
35         clock = 0;
36         forever #5 clock = ~clock;
37     end
38
39     initial begin
40         $monitor("lost(%d), lives(%d)", lost, lives);
41         ball_top = 9'd150;
42         load = 1'b1
43         @(posedge clock);
44         @(posedge clock);
45         ball_top = 9'd450;
46         @(posedge clock);
47         @(posedge clock);
48         @(posedge clock);
49     end
50
51 endmodule: ScoreCount_test
52
53 // Test brick functions
54 module brick_test();
55     logic exists, clock, reset;
56     logic [8:0] row, brick_top, brick_height,
57     logic [9:0] col, brick_left, brick_width,
58     logic atBrick;
59
60     Brick m(.*);
61
62     initial begin
63         clock = 0;
64         forever #5 clock = ~clock;
65     end
66
67     initial begin
68         $monitor("atBrick(%d), exists(%d)", atBrick, exists);
69         row = 9'd179;
```

```
70     col = 10'd245;
71     brick_top = 9'd150;
72     brick_height = 9'd30;
73     brick_left = 10'd90;
74     brick_width = 10'd100;
75     hit = 1'b1;
76     reset = 1'b1;
77     @(posedge clock);
78     @(posedge clock);
79     reset = 1'b0;
80     @(posedge clock);
81     @(posedge clock);
82     @(posedge clock);
83     end
84
85
86 endmodule: brick_test
87
88 // Test wall functions
89 module walls_test();
90     logic [8:0] row;
91     logic [9:0] col;
92     logic atWall, twall, lfwall;
93
94     Brick m(.*);
95
96     initial begin
97         clock = 0;
98         forever #5 clock = ~clock;
99     end
100
101     initial begin
102         $monitor("atWall(%d), twall(%d), lfwall(%d)", atWall, twall, lfwall);
103         row = 9'd179;
104         col = 10'd245;
105         @(posedge clock);
106         @(posedge clock);
107         row = 9'd459;
108         col = 10'd625;
109         @(posedge clock);
110         row = 9'd44;
111         col = 10'd325;
112         @(posedge clock);
113         row = 9'd44;
114         col = 10'd625;
115         @(posedge clock);
116     end
117
118 endmodule: walls_test
```

Lab Code [10 points]
Filename: vga_interface.sv
AndrewID: wenzhuo1

```
1 `default_nettype none
2
3 module vga
4   (input logic      CLOCK_50, reset,
5    output logic     HS, VS, blank,
6    output logic [8:0] row,
7    output logic [9:0] col);
8
9   logic clear_hs_ct, clear_vs_ct, two_clocks, start_col, start_row;
10  logic [10:0] hs_num_clk;
11  logic [19:0] vs_num_clk;
12
13
14
15  // counter to keep track of num clock period elapsed
16  Counter #(11) hs_ct (.en(1'b1), .clear(reset), .load(clear_hs_ct), .D(11'd0),
17    .up(1'b1), .clock(CLOCK_50), .Q(hs_num_clk));
18
19  Counter #(20) vs_ct(.en(1'b1), .clear(reset), .load(clear_vs_ct), .D(20'd0),
20    .up(1'b1), .clock(CLOCK_50), .Q(vs_num_clk));
21
22
23  // Send clear signal to counter every sync pulse period
24  MagComp #(11) HS_num_clock (.A(hs_num_clk), .B(11'd1599),
25    .AeqB(clear_hs_ct));
26  MagComp #(20) VS_num_clock (.A(vs_num_clk), .B(20'd833599),
27    .AeqB(clear_vs_ct));
28
29  // row counter
30    // increment on each clear
31  logic start_row_clr, start_col_clr, load_row, load_col;
32  assign start_row_clr = ~start_row;
33  assign start_col_clr = ~start_col;
34  assign load_row = ~VS;
35  assign load_col = ~HS;
36  Counter #(9) row_ct(.en(clear_hs_ct), .clear(start_row_clr), .up(1'b1),
37    .load(load_row), .D(9'd0), .clock(CLOCK_50), .Q(row));
38  // col counter
39    // increament every two clock
40  Counter #(10) col_ct(.en(two_clocks), .clear(start_col_clr), .up(1'b1),
41    .load(load_col), .D(10'd0), .clock(CLOCK_50), .Q(col));
42
43  assign two_clocks = hs_num_clk[0];
44
45  // HS/VS window check
46  range_check #(11) hs_period(.val(hs_num_clk),
47    .low(11'd192),
48    .high(11'd1599),
49    .is_between(HS));
50
51  range_check #(20) vs_period(.val(vs_num_clk),
52    .low(20'd3200),
53    .high(20'd833599),
54    .is_between(VS));
55
56  // Tdisp window check
57  range_check #(11) tdisphs_period(.val(hs_num_clk),
58    .low(11'd288),
59    .high(11'd1567),
60    .is_between(start_col));
61
62  range_check #(20) tdispvs_period(.val(vs_num_clk),
63    .low(20'd49600),
64    .high(20'd817599),
65    .is_between(start_row));
66
67  assign blank = ~(start_col & start_row);
68
69
```



```
70  
71 endmodule: vga
```